

Отчёт по задаче «КА-группировки»

Автор: Речкин Вадим Сергеевич

Группа: 23Б08-мм

GitHub: github.com/VadosikRRR/.../ka_groupings.cpp

1. Формулировка задачи

Есть n космических аппаратов (КА) и m связей между ними. Для каждой связи между КА u и v задан интервал активности $[t_{start}, t_{end})$ на общей шкале времени.

Задан глобальный интервал времени $[0, T)$ минут. Нужно на каждом такте длиной 10 минут формировать группировки КА.

Правило группировки на такте $[t, t + 10)$: строим граф только из тех связей, интервалы которых пересекаются с текущим тактом. Компоненты связности этого графа считаются группировками КА.

Входные данные

- n, m, T ;
- далее m строк: $u \ v \ t_{start} \ t_{end}$.

Выходные данные

Для каждого такта вывести его интервал времени и список группировок (каждая группировка — список номеров КА).

2. Идея решения

Для каждого такта $[t, t + 10)$:

1. очищаем список смежности;
2. для каждой связи проверяем пересечение интервалов:

$$[t, t + 10) \cap [l, r) \neq \emptyset \iff \max(t, l) < \min(t + 10, r);$$

3. если связь активна, добавляем ребро в граф текущего такта;
4. запускаем BFS/DFS по графу и находим компоненты связности;
5. печатаем найденные компоненты как группировки.

3. Код программы

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u;
    int v;
    int l;
    int r;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ifstream fin("example_input.txt");

    int n, m, T;
    fin >> n >> m >> T;

    vector<Edge> edges(m);
    vector<vector<int>> adj(n + 1);

    for (int i = 0; i < m; ++i) {
        fin >> edges[i].u >> edges[i].v >> edges[i].l >> edges[i].r;
    }

    const int STEP = 10;

    for (int t = 0; t < T; t += STEP) {
        for (int i = 1; i <= n; ++i) {
            adj[i].clear();
        }

        for (int i = 0; i < m; ++i) {
            if (max(t, edges[i].l) < min(t + STEP, edges[i].r)) {
                int u = edges[i].u;
                int v = edges[i].v;
                adj[u].push_back(v);
                adj[v].push_back(u);
            }
        }

        vector<int> used(n + 1, 0);
        vector<vector<int>> groups;

        for (int start = 1; start <= n; ++start) {
            if (used[start]) continue;

            vector<int> comp;
            queue<int> q;
            q.push(start);
```

```

        used[start] = 1;

        while (!q.empty()) {
            int v = q.front();
            q.pop();
            comp.push_back(v);

            for (int to : adj[v]) {
                if (!used[to]) {
                    used[to] = 1;
                    q.push(to);
                }
            }
        }

        sort(comp.begin(), comp.end());
        groups.push_back(comp);
    }

    sort(groups.begin(), groups.end());

    cout << "Tick [" << t << ", " << min(t + STEP, T) << "]" << '\n';
    for (int i = 0; i < (int)groups.size(); ++i) {
        cout << "Group " << (i + 1) << ":";
        for (int sat : groups[i]) {
            cout << ' ' << sat;
        }
        cout << '\n';
    }
    cout << '\n';
}

return 0;
}

```

4. Оценка сложности

Пусть $q = \lceil T/10 \rceil$ — число тактов.

На каждом такте:

- просмотр всех связей: $O(m)$,
- поиск компонент связности: $O(n + m)$.

Итог по времени:

$$O(q \cdot (n + m)).$$

Память:

- хранение рёбер: $O(m)$,
- список смежности текущего такта: $O(n + m)$,

- служебные массивы: $O(n)$.

Итог по памяти:

$$O(n + m).$$

5. Пример

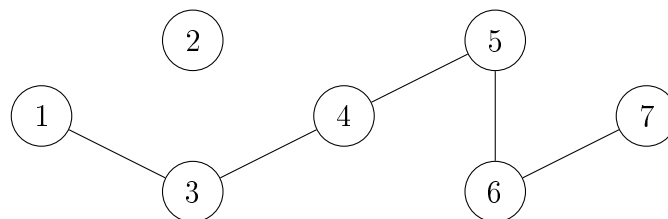
Вход:

```
7 7 60
1 2 0 25
2 3 5 30
4 5 10 45
5 6 20 55
6 7 35 60
1 3 15 40
3 4 30 50
```

Фрагмент результата:

```
Tick [0, 10)
Group 1: 1 2 3
Group 2: 4
Group 3: 5
Group 4: 6
Group 5: 7
```

Иллюстрация (граф активных связей на такте [30,40)):



Здесь все КА входят в одну компоненту связности, значит на этом такте образуется одна общая группировка.